

Q1 KNAPSACK

$$v[i, j] = \begin{cases} v[i-1, j] & \text{if } w_i > j \\ \max[v[i-1, j], v_i + v[i-1, j-w_i]] & \text{if } w_i \leq j \end{cases}$$

Capacity $w = 200$

Number of items $n = 7$

i	item	w_i	v_i
1	sofa set	80	120
2	LED TV	70	90
3	Refrigerator	60	150
4	Double bed	70	100
5	Laptop	5	70
6	Dinner set	15	40
7	Clothes Box	10	30

Dry Run :

Capacities considered = 0, 50, 100, 150, 200

w	0	50	100	150	200
$v[0, w]$	0	0	0	0	0
$v[1, w]$	0	0	120	120	120
$v[2, w]$	0	90	210	210	210
$v[3, w]$	0	90	210	240	360
$v[4, w]$	0	90	210	240	360
$v[5, w]$	0	160	280	310	430
$v[6, w]$	0	200	320	350	470
$v[7, w]$	0	230	350	380	500

Answer $\rightarrow v[7, 200] = 500$

Dry Run:

$\rightarrow$ Row $i=0 \rightarrow$ No items

$\rightarrow$ Row $i=1$ (Sofa: 80, 120)

• For $w < 80 \rightarrow$ cannot take sofa

• For $w \geq 80 \rightarrow$ take sofa

$\rightarrow$ Row $i=2$ (LED TV: 20, 90)

• $v[2, 100] = \max(120, 90 + v[1, 80])$

$= \max(120, 210) = 210$

$\rightarrow$ Row $i=3$ (Refrigerator: 60, 150)

• $v[3, 150] = \max(210, 150 + v[2, 90]) = \max(210, 240)$

• $v[3, 200] = \max(210, 150 + v[2, 140]) = \max(210, 360)$

$\rightarrow$ Row $i=4$ (Double Bed: 70, 100)

• $v[4, 200] = \max(360, 100 + v[3, 130])$

$= \max(360, 310)$

$= 360$

$\rightarrow$ Row $i=5$ (Laptop: 5, 70)

• $v[5, 200] = \max(360, 70 + v[4, 195]) = 430$

$\rightarrow$ Row $i=6$ (Dinner set: 15, 40)

• $v[6, 200] = \max(430, 40 + v[5, 185])$

$= 470$

$\rightarrow$ Row $i=7$ (Clothes Box: 10, 30)

• $v[7, 200] = \max(470, 30 + v[6, 190])$

$= 500$

Q2 Max Heap

```
BuildHeap (Array A, int n) {  
    for (i = n/2 ; i > 0 ; i--)  
    {  
        Heapify (A, i, n);  
    }  
}
```

```
Heapify (Arr A, int i, int m) {  
    L = left (i) ;  
    r = right (i) ;  
    max = i ;  
  
    if (L ≤ m && A[L] > A[max])  
        max = L ;  
  
    if (r ≤ m && A[r] > A[max])  
        max = r ;  
  
    if (max ≠ i) {  
        swap (A[i], A[max]) ;  
        Heapify (A, max, m) ;  
    }  
}
```

Incoming Calls (priorities): 25, 40, 10, 60, 35

Build Heap Dry-Run

Initial array

Index :	1	2	3	4	5
A :	25	40	10	60	35

→ Step 1: $i = \lfloor \frac{5}{2} \rfloor = 2$

- Node = 40

- Children = 60, 35

- Max = 60 → SWAP

25 60 10 40 35

→ Step 2: $i = 1$

- Node = 25

- Children = 60, 10

- Max = 60 → SWAP

60 25 10 40 35

→ Heapify → $i = 2$

- Node = 25

- Children = 40, 35

- Max = 40 → SWAP

60 40 10 25 35

Dispatch Order (Extract Max)

1. 60 → Cardiac Arrest
2. 40 → Severe accident
3. 35 → Fire incident
4. 25 → injury
5. 10 → minor issue

60 → 40 → 35 → 25 → 10

Time Complexity

- KNAPSACK: $O(nw)$

- Heap Operations: $O(\log n)$